Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 5

Due date: Monday 11 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Parallel Space Solution of a nonlinear PDE using MPI [in total 60 points]

1.1. Initialize/finalize MPI and welcome message [5 Points]

First we are asked to complete the initialization and finalization of MPI in the main function, and to print a welcome message with the simulation parameters in the file `main.cpp`.
In listings 1 we can see the initialization of MPI. We initialize MPI with `MPI_Init`, and then we query the number of processes and the rank of each process with `MPI_Comm_size` and `MPI_Comm_rank`. We store these values in the `domain` object, which is defined in the file `data.h`.

Listing 1: Initialization of MPI

```
1     MPI_Init(&argc, &argv);
2
3     int size = 1, rank = 0;
4     MPI_Comm_size(MPI_COMM_WORLD, &size);
5     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
6
7     domain.rank = rank;
8     domain.size = size;
9
10    // read command line arguments
11    readcmdline(options, argc, argv);
12
13    // initialize sub-domain (data.{h,cpp})
14    domain.init(rank, size, options);
15    int nx = domain.nx;
16    int ny = domain.ny;
17    int N  = domain.N;
18    int nt  = options.nt;
```

In listing 2 we can see the finalization of MPI. We free the Cartesian communicator that we created in the initialization phase, and then we call `MPI_Finalize` to clean up all MPI resources.

Listing 2: Finalization of MPI

```
1     // finalize MPI
2     MPI_Comm_free(&domain.comm_cart);
3     MPI_Finalize();
```

In listing 3 we can see the welcome message that we print at the beginning of the simulation. We only print this message from the rank 0 process, to avoid cluttering the output.

Listing 3: Welcome message

```
1 if(rank == 0)
2     {
3         std::cout << std::string(80, '=') << std::endl;
4         std::cout << "                      Welcome to mini-stencil!" <<
              std::endl;
5         std::cout << "version   :: C++ MPI" << std::endl;
6         std::cout << "processes :: " << size << std::endl;
7         std::cout << "mesh      :: " << options.nx << " * " << options.nx
8                                      << " dx = " << options.dx << std::
                                         endl;
9         std::cout << "time      :: " << nt << " time steps from 0 .. "
10                                        << options.nt*options.dt << std
                                           ::endl;
11        std::cout << "iteration :: " << "CG "          << max_cg_iters
12                                     << ", Newton "    << max_newton_iters
13                                     << ", tolerance " << tolerance << std
                                         ::endl;
14        std::cout << std::string(80, '=') << std::endl;
```

```
15      }
```

1.2. Domain decomposition [10 Points]

Here we are asked to design a domain decomposition strategy that is compatible with any square grid size and any number of MPI processes while considering load balancing, communication overhead and computational efficiency. We were suggested to look at the short discussion in [1, Sec. 10.4.1].
We can first notice that for a subdomain of size $n_x \times n_y$ we have a total computation of $n_x \cdot n_y$ while the halo exchange requires $2 \cdot (n_x + n_y)$ communicated boundary points. For a fixed local area $A = n_x n_y$, the arithmetic-geometric mean inequality gives

$$n_x + n_y \geq 2\sqrt{n_x n_y} = 2\sqrt{A},$$

with equality when $n_x = n_y$. Therefore, for a fixed local area, communication is minimized when the rectangle is as close to a square as possible. Indeed in [1, Sec. 10.4.1] they suggest that we have to avoid slab decomposition. For example, if you split only in x, each process has a long thin strip. It computes fewer points, but the communicated boundary length stays large. A 2D Cartesian decomposition is better because both n_x and n_y decrease as the number of MPI processes increases.

Listing 4: Initialization of the sub-domain

```
1  void SubDomain::init(int mpi_rank, int mpi_size,
2                       Discretization& discretization) {
3      // Determine the number of sub-domains in the x and y dimensions
4      //       using MPI_Dims_create
5      int dims[2] = {0, 0};
6      MPI_Dims_create(mpi_size,2,dims);
7      ndomy = dims[0];
8      ndomx = dims[1];
9
10     // Create a 2D non-periodic Cartesian topology using MPI_Cart_create
11     int periods[2] = {0, 0};
12     MPI_Cart_create(MPI_COMM_WORLD,2,dims,periods,0,&comm_cart);
13
14     // Retrieve coordinates of the rank in the topology using
15     // MPI_Cart_coords
16     int coords[2] = {0, 0};
17     MPI_Cart_coords(comm_cart, mpi_rank , 2 , coords);
18     domy = coords[0] + 1;
19     domx = coords[1] + 1;
20
21     // Set neighbours for all directions using MPI_Cart_shift
22     MPI_Cart_shift(comm_cart, 0 , 1 , &neighbour_south, &neighbour_north);
23     MPI_Cart_shift(comm_cart, 1 , 1 , &neighbour_west, &neighbour_east);
24
25     // get Cartesian coordinates, but use 0-based versions for
           computations
26     int coord_y = coords[0];
27     int coord_x = coords[1];
28
29     // distribute grid points in x direction
30     int base_nx = discretization.nx / ndomx;
31     int rem_x   = discretization.nx % ndomx;
32
33     nx = base_nx + (coord_x < rem_x ? 1 : 0);
34
35     startx = coord_x * base_nx + std::min(coord_x, rem_x);
36     endx   = startx + nx - 1;
37
38     // distribute grid points in y direction
39     int base_ny = discretization.nx / ndomy;
```

```
    int rem_y   = discretization.nx % ndomy;

    ny = base_ny + (coord_y < rem_y ? 1 : 0);

    starty = coord_y * base_ny + std::min(coord_y, rem_y);
    endy   = starty + ny - 1;

    // get total number of grid points in this sub-domain
    N = nx*ny;

    rank = mpi_rank;
    size = mpi_size;
}
```

This code splits the global $N \times N$ grid into a 2D Cartesian process grid. The total number of points in each direction is divided by the number of processes along that axis, giving a base block size (`base_nx` and `base_ny`). The remainders (`rem_x` and `rem_y`) are assigned one extra grid point to the first ranks in each direction, so the subdomains differ in size by at most one cell. The expressions for `startx`, `endx`, `starty`, and `endy` then compute the global index range of each local block using prefix sums. This keeps the workload balanced while still covering the full domain without overlaps or gaps.

1.3. Linear algebra kernels [5 Points]

The linear algebra kernels were adapted to the MPI domain-decomposed setting. Most vector operations, such as `hpc_fill`, `hpc_axpy`, `hpc_scale`, `hpc_lcomb`, `hpc_copy`, `hpc_scaled_diff`, and `hpc_add_scaled_diff`, remain local. Each MPI process stores only its own subdomain, and these routines operate independently on the local entries.

The only kernels requiring MPI communication are the reduction operations `hpc_dot` and `hpc_norm2`. In both cases, each process first computes its local contribution. The local results are then combined across all MPI ranks using `MPI_Allreduce` with `MPI_SUM`. This is necessary because scalar products and norms are global quantities over the full distributed vector. In listing 5 we can see the implementation of the dot product. Each process computes the local dot product of its subdomain, and then we call `MPI_Allreduce` to sum up the local results into a global result. The same pattern is used for the 2-norm in listing 6, where we compute the local sum of squares and then take the square root of the global sum.

Listing 5: MPI parallel implementation of the dot product

```
double hpc_dot(Field const& x, Field const& y) {
    int N = y.length();
    double result = 0;
    // compute local result
    for (int i = 0; i < N; i++) {
        result += x[i] * y[i];
    }

    double global_result = 0.0;
    MPI_Allreduce(&result, &global_result, 1, MPI_DOUBLE, MPI_SUM,
                  data::domain.comm_cart);

    return global_result;
}
```

Listing 6: MPI parallel implementation of the 2-norm

```
double hpc_norm2(Field const& x) {
    int N = x.length();
    double result = 0;
    // compute local result
    for (int i = 0; i < N; i++) {
        result += x[i] * x[i];
    }
```

```
8
9       double global_result = 0.0;
10      MPI_Allreduce(&result, &global_result, 1, MPI_DOUBLE, MPI_SUM,
11                    data::domain.comm_cart);
12
13      return sqrt(global_result);
14  }
```

The conjugate-gradient solver `hpc_cg` did not require structural changes, since it calls the linear algebra kernels internally. Once the dot products and norms are made global, the CG coefficients are computed consistently across all ranks.

1.4. The diffusion stencil: Ghost cells exchange [10 Points]

Here we are asked to implement the ghost cell exchange between neighboring processes using non- blocking point-to-point communication with the objective to overlap computation and communication.
In listing 7 we provide a piece of the `diffusion` method where we tried to implement the ghost cell exchange.

Listing 7: Ghost cell exchange using non-blocking point-to-point communication

```
1       int countx = nx ; // number of interior points in x for rows
2       int county = ny ; // number of interior points in y for columns
3
4       // temporary send buffers for columns (non-contiguous in Field)
5       std::vector<double> sendW(county), sendE(county);
6       for (int j = 0; j < county; ++j) {
7           sendW[j] = s_new(0, j);          // left interior column
8           sendE[j] = s_new(nx-1, j);      // right interior column
9       }
10
11      MPI_Request reqs[8];
12      int reqcnt = 0;
13
14      // Post non-blocking receives into the boundary arrays (contiguous)
15      if (domain.neighbour_north != MPI_PROC_NULL) {
16          MPI_Irecv(&bndN[0], countx, MPI_DOUBLE, domain.neighbour_north, 0,
17                    domain.comm_cart, &reqs[reqcnt++]);
18      }
19      if (domain.neighbour_south != MPI_PROC_NULL) {
20          MPI_Irecv(&bndS[0], countx, MPI_DOUBLE, domain.neighbour_south, 0,
21                    domain.comm_cart, &reqs[reqcnt++]);
22      }
23      if (domain.neighbour_west != MPI_PROC_NULL) {
24          MPI_Irecv(&bndW[0], county, MPI_DOUBLE, domain.neighbour_west, 1,
25                    domain.comm_cart, &reqs[reqcnt++]);
26      }
27      if (domain.neighbour_east != MPI_PROC_NULL) {
28          MPI_Irecv(&bndE[0], county, MPI_DOUBLE, domain.neighbour_east, 1,
29                    domain.comm_cart, &reqs[reqcnt++]);
30      }
31
32      // Post non-blocking sends
33      if (domain.neighbour_north != MPI_PROC_NULL) {
34          MPI_Isend(&s_new(0, ny-1), countx, MPI_DOUBLE, domain.
                neighbour_north, 0,
35                    domain.comm_cart, &reqs[reqcnt++]);
36      }
37      if (domain.neighbour_south != MPI_PROC_NULL) {
38          MPI_Isend(&s_new(0, 0), countx, MPI_DOUBLE, domain.neighbour_south
                , 0,
39                    domain.comm_cart, &reqs[reqcnt++]);
40      }
```

```
41      if (domain.neighbour_west != MPI_PROC_NULL) {
42          MPI_Isend(sendW.data(), county, MPI_DOUBLE, domain.neighbour_west,
                1,
43                    domain.comm_cart, &reqs[reqcnt++]);
44      }
45      if (domain.neighbour_east != MPI_PROC_NULL) {
46          MPI_Isend(sendE.data(), county, MPI_DOUBLE, domain.neighbour_east,
                1,
47                    domain.comm_cart, &reqs[reqcnt++]);
48      }
49
50      // Wait for all posted requests
51      if (reqcnt > 0) MPI_Waitall(reqcnt, reqs, MPI_STATUSES_IGNORE);
```

This implementation exchanges the halo cells by separating the communication into two directions. The north and south boundaries can be sent directly, because the corresponding rows are contiguous in memory, while the west and east boundaries require temporary buffers since the columns of the local field are not stored contiguously. For this reason, first we pack the left and right interior columns into `sendW` and `sendE`, and then posts non-blocking receives and sends for all available neighbors. Although the exchange uses non-blocking MPI calls, in the current implementation the code waits immediately after posting the sends and receives. Therefore, the implementation avoids deadlock and prepares the code structure for overlap, but it does not yet realize a true overlap between communication and stencil computation. The tags `0` and `1` distinguish row and column messages, and the checks against `MPI_PROC_NULL` ensure that boundary processes simply skip the missing neighbors. Finally, `MPI_Waitall` guarantees that all halo exchanges are complete before the updated ghost layers are used in the next stencil step.

1.5. Implement parallel I/O [10 Points]

By looking at the file `mpio_io.c` we were asked to implement the output of the computed solution with MPI-I/O.
This was done by completing the function `write_binary` in `main.cpp` as shown in listing 8.

Listing 8: MPI-IO implementation of the output

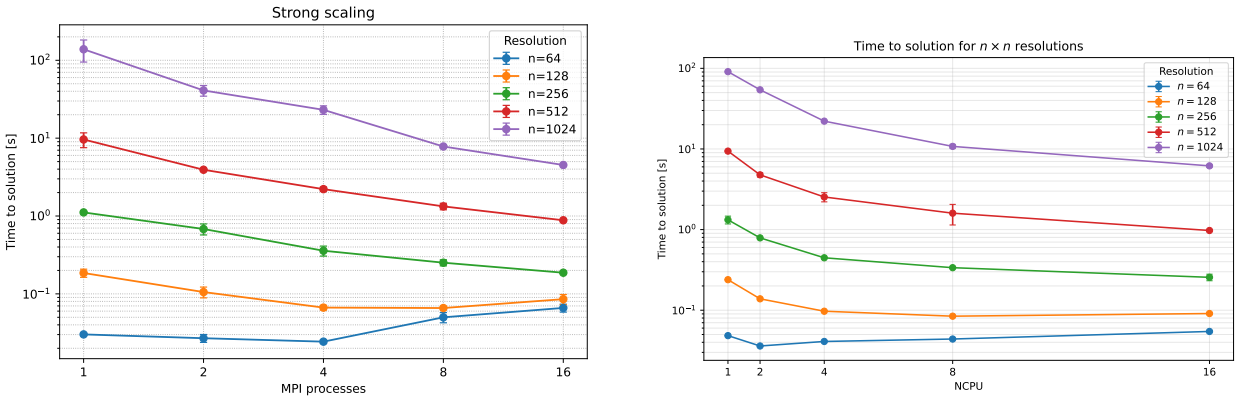
```
1  void write_binary(std::string fname, Field &u, SubDomain &domain,
2                    Discretization &options) {
3      MPI_File file;
4      MPI_File_open(MPI_COMM_WORLD, fname.c_str(),
5                    MPI_MODE_CREATE | MPI_MODE_WRONLY, MPI_INFO_NULL, &file)
                         ;
6
7      // global sizes (full global grid)
8      int sizes[2] = { options.nx, options.nx };
9      // local subarray sizes
10     int subsizes[2] = { domain.ny, domain.nx };
11     // start indices in the global array
12     int starts[2] = { domain.starty, domain.startx };
13
14     MPI_Datatype filetype;
15     MPI_Type_create_subarray(2, sizes, subsizes, starts, MPI_ORDER_C,
16                              MPI_DOUBLE, &filetype);
17     MPI_Type_commit(&filetype);
18
19     MPI_Offset disp = 0;
20     MPI_File_set_view(file, disp, MPI_DOUBLE, filetype, "native",
           MPI_INFO_NULL);
21
22     // write the local data collectively
23     MPI_File_write_all(file, u.data(), domain.N, MPI_DOUBLE,
           MPI_STATUS_IGNORE);
24
25     MPI_Type_free(&filetype);
```

```
26      MPI_File_close(&file);
27  }
```

The implementation uses MPI-IO to write the distributed field data into a single binary file. Each process defines a subarray datatype that describes its local portion of the global grid, and then collectively writes its data using `MPI_File_write_all`. The use of `MPI_Type_create_subarray` allows each process to specify its unique offset and size within the global array, enabling a seamless parallel output without the need for gathering data on a single rank.

1.6. Strong scaling [10 Points]

Here we are asked to plot the time to solution for NCPU $= 1, 2, 4, 8, 16$ processes across resolutions of $n \times n$, where $n = 64, 128, 256, 512, 1024$. Figure 1 shows the strong scaling behaviour of the MPI implementation



(a) Strong scaling results for the time to solution in the MPI implementation.

(b) Strong scaling results for the time to solution in the OpenMP implementation.

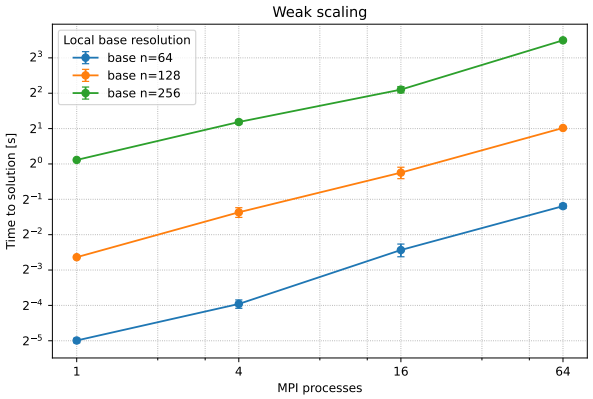
Figure 1: Strong scaling results for the time to solution.

and compares it with the OpenMP implementation from Project 3. For the larger resolutions, especially $n = 512$ and $n = 1024$, the MPI implementation shows a clear reduction of the time to solution when increasing the number of processes. This is the expected behaviour in a strong scaling experiment: the global problem size is fixed, but the work is distributed among more processes. The improvement is particularly visible for the largest grid, where the computational work per process is still large enough to compensate for the cost of MPI communication.

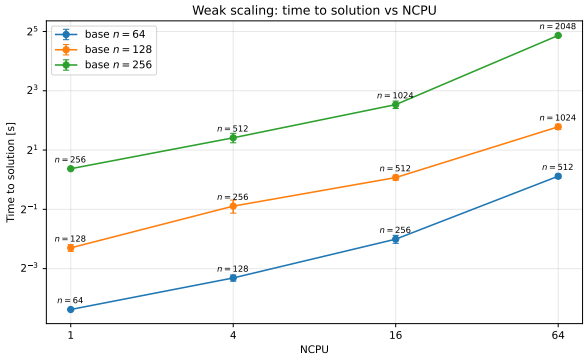
For small resolutions, for example $n = 64$ and partly $n = 128$, the scaling is much weaker and can even become non-monotonic. This is expected because the amount of computation per process is very small. In this regime, the overhead due to MPI initialization, halo exchanges, global reductions in the conjugate-gradient solver, and synchronization becomes comparable to the actual stencil computation. Therefore, adding more processes does not necessarily reduce the runtime. For $n = 64$, the runtime even increases when many processes are used, which indicates that the problem is too small to benefit from parallelization.

Compared with the OpenMP implementation, the qualitative behaviour is similar: large resolutions scale better, while small resolutions are dominated by parallel overhead. However, the reason for the overhead is different. In OpenMP, all threads run in shared memory, so there is no explicit halo exchange between processes, but the code can suffer from thread-management overhead, synchronization, cache effects, and memory-bandwidth contention. In the MPI version, each process owns only a subdomain and communicates explicitly with neighbouring processes, so the main overhead comes from halo exchanges and global reductions. For large enough problems, the MPI implementation scales well because the local computation dominates the communication cost. For small problems, both implementations are limited by overhead rather than by floating-point work.

Overall, the strong scaling results confirm that the MPI implementation is effective for sufficiently large grids, while small grids are not suitable for strong scaling because the local workload becomes too small.



(a) Weak scaling results for the time to solution.



(b) Weak scaling results for the time to solution vs. number of processes.

Figure 2: Weak scaling results for the time to solution.

1.7. Weak scaling [10 Points]

Figure 2 shows the weak scaling behaviour. In a weak scaling experiment, the problem size is increased together with the number of processes in such a way that the amount of work per process should remain approximately constant. Ideally, the time to solution would therefore remain constant as the number of MPI processes increases.

The measured results show that the time to solution increases with the number of processes for all base resolutions. This means that the weak scaling is not ideal. The increase is caused by the fact that, although the local computational work per process is kept approximately constant, the communication and synchronization costs grow when more processes are used. In particular, the halo exchange involves more subdomain boundaries, and the conjugate-gradient method requires global reductions, whose cost increases with the number of MPI ranks. These global reductions are especially important because they introduce synchronization across all processes.

The weak scaling behaviour is better for smaller base resolutions in absolute time, but the relative increase is still visible. For larger base resolutions, such as base $n = 256$, the runtime is higher because each process has a larger local problem. The growth of the runtime with $NCPU$ shows that communication and synchronization become increasingly important as the global problem size and the number of processes increase.

Compared with the OpenMP implementation of Project 3, the MPI implementation again shows the same general trend: weak scaling is not perfectly flat. However, the source of the loss of efficiency is different. In OpenMP, the threads share the same memory space, so there is no explicit message passing, but the performance can be limited by memory bandwidth, cache/NUMA effects, and synchronization inside the iterative solver. In MPI, the memory is distributed, which improves memory locality, but this comes at the price of explicit communication between neighbouring subdomains and collective reductions. Therefore, the MPI weak scaling is mainly limited by communication and global synchronization, while the OpenMP weak scaling is mainly limited by shared-memory contention and thread synchronization.

Overall, the weak scaling results are reasonable for a distributed-memory stencil code. The runtime does not remain perfectly constant, but the increase is expected because the PDE solver is not purely local: every timestep and every conjugate-gradient iteration require communication between subdomains and global reductions.

1.8. Bonus [20 Points]: Overlapping communication/computation details

2. Python for High-Performance Computing [in total 40 points]

2.1. Sum of ranks: MPI collectives [5 Points]

We were asked to implement the sum of ranks using MPI for Python's collective communication method.

2.1.1. Pickle based

In the first case we had to use pickle-based communication of generic Python objects, i.e. the all-lowercase methods.
In listing 9 we can see the implementation of the sum of ranks using the pickle-based collective communication method.

Listing 9: Sum of ranks using pickle-based collective communication

```
1  from mpi4py import MPI
2
3  # get comm, size, rank & host name
4  comm = MPI.COMM_WORLD
5  size = comm.Get_size()
6  rank = comm.Get_rank()
7  proc = MPI.Get_processor_name()
8
9  # pickle-based
10 total = comm.allreduce(rank, op=MPI.SUM)
11
12 print(
13     f"Sum is {total} on processor {proc}, "
14     f"rank {rank} out of {size} processes"
15 )
```

The pickle-based implementation uses the lower-case collective routine `comm.allreduce`. Each process contributes its local Python integer `rank`, and the operation `MPI.SUM` combines all contributions into the global sum. Since `allreduce` is a collective operation, the result is returned to every MPI process. The lower-case mpi4py interface works with generic Python objects and therefore uses pickle serialization internally. This makes the implementation very compact, although it may introduce more overhead than direct buffer communication for large data objects.

Listing 10: Output of the sum of ranks using pickle-based collective communication

```
1 ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/01_sum$ srun
     python3 sum_pickle.py
2 Sum is 28 on processor eu-a2p-301, rank 2 out of 8 processes
3 Sum is 28 on processor eu-a2p-301, rank 3 out of 8 processes
4 Sum is 28 on processor eu-a2p-299, rank 0 out of 8 processes
5 Sum is 28 on processor eu-a2p-344, rank 6 out of 8 processes
6 Sum is 28 on processor eu-a2p-335, rank 5 out of 8 processes
7 Sum is 28 on processor eu-a2p-299, rank 1 out of 8 processes
8 Sum is 28 on processor eu-a2p-344, rank 7 out of 8 processes
9 Sum is 28 on processor eu-a2p-335, rank 4 out of 8 processes
```

The output confirms that all MPI processes compute the same final sum. In this run, `size = 8`, therefore the expected result is

$$0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 = 28.$$

Every rank prints `Sum is 28`, showing that the ring communication has circulated all rank values correctly through the communicator.

2.1.2. Direct array data communication

In the second case we had to use direct array data communication with the all-uppercase methods, i.e. the MPI methods that directly handle array data without pickling.
In listing 11 we can see the implementation of the sum of ranks using direct array data communication.

Listing 11: Sum of ranks using direct array data communication

```
from mpi4py import MPI
import numpy as np

# get comm, size, rank & host name
comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
proc = MPI.Get_processor_name()

# direct-buffer collective communication
sendbuf = np.array(rank, dtype=np.int32)
recvbuf = np.array(0, dtype=np.int32)

comm.Allreduce(sendbuf, recvbuf, op=MPI.SUM)

print(
    f"Sum is {recvbuf.item()} on processor {proc}, "
    f"rank {rank} out of {size} processes"
)
```

The direct-buffer implementation uses the upper-case collective routine `comm.Allreduce`. In contrast to the lower-case `allreduce`, this routine operates directly on buffer-provider objects, here NumPy arrays. Each process stores its local contribution in the array `sendbuf`, while the global sum is written into `recvbuf`. Since the data are communicated as raw memory buffers with an explicit NumPy datatype, mpi4py can avoid Python object serialization and use a communication path closer to the underlying C MPI interface.

Listing 12: Output of the sum of ranks using direct array data communication

```
ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/01_sum$ srun
   python3 sum_fast.py
Sum is 28 on processor eu-a2p-335, rank 4 out of 8 processes
Sum is 28 on processor eu-a2p-335, rank 5 out of 8 processes
Sum is 28 on processor eu-a2p-301, rank 2 out of 8 processes
Sum is 28 on processor eu-a2p-299, rank 0 out of 8 processes
Sum is 28 on processor eu-a2p-344, rank 6 out of 8 processes
Sum is 28 on processor eu-a2p-301, rank 3 out of 8 processes
Sum is 28 on processor eu-a2p-299, rank 1 out of 8 processes
Sum is 28 on processor eu-a2p-344, rank 7 out of 8 processes
```

The result is again correct: for eight MPI processes the sum of ranks is equal to 28, and all ranks obtain the same value. This shows that the direct-buffer implementation is logically equivalent to the pickle-based version.

2.2. Ghost cell exchange between neighboring processes [5 Points]

Here we were asked to create a two-dimensional, periodic Cartesian topology and implement ghost cell exchange.
In listing 13 we can see the implementation of the ghost cell exchange between neighboring processes in Python using mpi4py.

Listing 13: Ghost cell exchange between neighboring processes in Python using mpi4py

```
from mpi4py import MPI
import numpy as np

# get comm, size, rank & host name
comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
proc = MPI.Get_processor_name()

SUBDOMAIN_SIZE = 6
```

```
DOMAIN_SIZE = SUBDOMAIN_SIZE + 2

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Initialization
data = np.full((DOMAIN_SIZE, DOMAIN_SIZE), -1, dtype=np.int32)
data[1:-1, 1:-1] = rank

dims = [3, 4]
periods = [True, True]

cart = comm.Create_cart(dims=dims, periods=periods, reorder=False)

rank_south, rank_north = cart.Shift(1, 1)
rank_west, rank_east   = cart.Shift(0, 1)

# North/South exchange
cart.Sendrecv(
    sendbuf=data[DOMAIN_SIZE-2, 1:-1],
    dest=rank_north,
    sendtag=0,
    recvbuf=data[0, 1:-1],
    source=rank_south,
    recvtag=0
)

cart.Sendrecv(
    sendbuf=data[1, 1:-1],
    dest=rank_south,
    sendtag=1,
    recvbuf=data[DOMAIN_SIZE-1, 1:-1],
    source=rank_north,
    recvtag=1
)

# West/East exchange: columns are not contiguous, so use temporary arrays
recv_west = np.empty(SUBDOMAIN_SIZE, dtype=np.int32)
recv_east = np.empty(SUBDOMAIN_SIZE, dtype=np.int32)

send_east = np.ascontiguousarray(data[1:-1, DOMAIN_SIZE-2])
send_west = np.ascontiguousarray(data[1:-1, 1])

cart.Sendrecv(
    sendbuf=send_east,
    dest=rank_east,
    sendtag=2,
    recvbuf=recv_west,
    source=rank_west,
    recvtag=2
)
data[1:-1, 0] = recv_west

cart.Sendrecv(
    sendbuf=send_west,
    dest=rank_west,
    sendtag=3,
    recvbuf=recv_east,
    source=rank_east,
```

```
71      recvtag=3
72  )
73  data[1:-1, DOMAIN_SIZE-1] = recv_east
```

Each MPI process owns a local subdomain of size `SUBDOMAIN_SIZE` by `SUBDOMAIN_SIZE`. The full local array `data` has size `DOMAIN_SIZE` by `DOMAIN_SIZE`, where `DOMAIN_SIZE = SUBDOMAIN_SIZE + 2`. The additional two rows and columns are the ghost layers. The interior entries `data[1:-1, 1:-1]` are initialized with the MPI rank of the process, while the ghost cells are initialized to `-1`. After the exchange, the ghost cells should contain the rank numbers of the neighbouring processes.

The Cartesian topology is created with `comm.Create_cart`. In this implementation, the process grid is fixed to `dims = [3, 4]`, meaning that the 12 MPI processes are arranged in a two-dimensional grid with 3 processes in one direction and 4 processes in the other. The array `periods = [True, True]` makes the topology periodic in both directions. Therefore, processes on the boundary of the process grid still have neighbours: for example, the process at the right boundary has as east neighbour the corresponding process at the left boundary.
The neighbours are obtained using `cart.Shift`. The call `cart.Shift(1, 1)` returns the neighbours in the second Cartesian direction, which are used here as the south and north neighbours. Similarly, `cart.Shift(0, 1)` returns the neighbours in the first Cartesian direction, used here as the west and east neighbours. This avoids manually computing neighbour ranks and makes the code independent of the rank ordering inside the Cartesian communicator.

The north/south exchange communicates rows. Since NumPy arrays are stored in row-major order, slices such as `data[DOMAIN_SIZE-2, 1:-1]` and `data[1, 1:-1]` are contiguous in memory. Therefore, they can be passed directly to `Sendrecv`. The last interior row is sent to the north neighbour and the first ghost row is filled with data received from the south neighbour. Conversely, the first interior row is sent to the south neighbour and the last ghost row is filled with data received from the north neighbour.
The west/east exchange communicates columns. In row-major storage, columns are not contiguous in memory. For this reason, the code first creates contiguous temporary send buffers using `np.ascontiguousarray`. The east interior column is copied into `send_east`, and the west interior column is copied into `send_west`. The received columns are stored in the temporary arrays `recv_west` and `recv_east`, and then copied into the corresponding ghost columns of `data`. This avoids problems associated with sending non-contiguous NumPy views directly.

The use of `Sendrecv` again prevents deadlocks. All processes participate in the same pairwise exchange pattern, and each call simultaneously sends one boundary buffer and receives the corresponding ghost buffer. Different message tags are used for the two directions of each exchange. This makes the matching of messages unambiguous and prevents a message from one exchange direction from being incorrectly matched with another one.

Listing 14: Output of the ghost cell exchange between neighboring processes in Python using mpi4py

```
1  ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/02_ghost$ srun
       python3 ghost.py
2  North rank 6, South rank 4, West rank 1, East rank 9
3  data of rank 5 after communication
4  j
5  ^
6  | -1.0  6.0  6.0  6.0  6.0  6.0  6.0 -1.0
7  |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
8  |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
9  |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
10 |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
11 |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
12 |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
13 | -1.0  4.0  4.0  4.0  4.0  4.0  4.0 -1.0
14 ----------------------------------------> i
```

The printed array corresponds to the local subdomain of rank 5 after the ghost-cell exchange. The interior of the array is filled with the value 5, because those entries belong to rank 5 itself. The ghost layers contain the ranks of the neighbouring processes in the Cartesian topology.

The top ghost row contains the value 6, which corresponds to the north neighbour of rank 5. The bottom ghost row contains the value 4, corresponding to the south neighbour. Similarly, the left ghost column

contains the value 1, and the right ghost column contains the value 9. This confirms that each process has correctly exchanged its boundary data with the four neighbouring processes.

The corner entries remain equal to `-1`. This is expected because the implemented exchange only communicates the north, south, east, and west neighbours required by a standard five-point stencil. Corner values would only be needed for a stencil involving diagonal neighbours, which is not the case here.

2.3. A self-scheduling example: Parallel Mandelbrot [30 Points]

2.3.1. Implementation

Here we were asked to implement the manager-worker pattern to compute the Mandelbrot set in parallel. The manager process is responsible for distributing the work and collecting the results, while the worker processes perform the actual computation of the Mandelbrot set for assigned tasks.
The implementation of the manager-worker pattern for computing the Mandelbrot set is provided in the listing 15.

Listing 15: Manager-worker pattern for computing the Mandelbrot set in parallel

```
# some parameters
MANAGER = 0       # rank of manager
TAG_TASK      = 1 # task       message tag
TAG_TASK_DONE = 2 # tasks done message tag
TAG_DONE      = 3 # done       message tag

def manager(comm, tasks):
    size = comm.Get_size()
    nworkers = size - 1

    next_task = 0
    active_workers = 0
    TasksDoneByWorker = {i: 0 for i in range(size)}

    # send initial tasks
    for worker in range(1, size):
        if next_task < len(tasks):
            comm.send((next_task, tasks[next_task]), dest=worker, tag=
               TAG_TASK)
            next_task += 1
            active_workers += 1
        else:
            comm.send(None, dest=worker, tag=TAG_DONE)

    # receive results and assign new tasks
    status = MPI.Status()

    while active_workers > 0:
        task_id, completed_task = comm.recv(
        source=MPI.ANY_SOURCE,
        tag=TAG_TASK_DONE,
        status=status
        )

        worker = status.Get_source()
        TasksDoneByWorker[worker] += 1

        # replace the completed task in the task list
        tasks[task_id] = completed_task

        if next_task < len(tasks):
            comm.send((next_task, tasks[next_task]), dest=worker, tag=
               TAG_TASK)
            next_task += 1
```

```
43          else:
44              comm.send(None, dest=worker, tag=TAG_DONE)
45              active_workers -= 1
46
47      return TasksDoneByWorker
48
49
50  def worker(comm):
51      while True:
52          status = MPI.Status()
53          message = comm.recv(source=MANAGER, tag=MPI.ANY_TAG, status=status
                )
54
55          if status.Get_tag() == TAG_DONE:
56              break
57
58          task_id, task = message
59          task.do_work()
60
61          comm.send((task_id, task), dest=MANAGER, tag=TAG_TASK_DONE)
62
63  def readcmdline(rank):
64      """
65      Read command line arguments
66
67      Parameters
68      ----------
69      rank : int
70          Rank of calling MPI process
71
72      Returns
73      -------
74      nx : int
75          number of gridpoints in x-direction
76      ny : int
77          number of gridpoints in y-direction
78      ntasks : int
79          number of tasks
80      """
81      # report usage
82      if len(sys.argv) != 4:
83          if rank == MANAGER:
84              print("Usage: manager_worker nx ny ntasks")
85              print("  nx     number of gridpoints in x-direction")
86              print("  ny     number of gridpoints in y-direction")
87              print("  ntasks number of tasks")
88          sys.exit()
89
90      # read nx, ny, ntasks
91      nx = int(sys.argv[1])
92      if nx < 1:
93          sys.exit("nx must be a positive integer")
94      ny = int(sys.argv[2])
95      if ny < 1:
96          sys.exit("ny must be a positive integer")
97      ntasks = int(sys.argv[3])
98      if ntasks < 1:
99          sys.exit("ntasks must be a positive integer")
100
101     return nx, ny, ntasks
```

```
102
103
104 if __name__ == "__main__":
105
106     # get COMMON WORLD communicator, size & rank
107     comm    = MPI.COMM_WORLD
108     size    = comm.Get_size()
109     my_rank = comm.Get_rank()
110
111     # report on MPI environment
112     if my_rank == MANAGER:
113         print(f"MPI initialized with {size:5d} processes")
114
115     # read command line arguments
116     nx, ny, ntasks = readcmdline(my_rank)
117
118     # start timer
119     timespent = - time.perf_counter()
120
121     if my_rank == MANAGER:
122         x_min = -2.
123         x_max = +1.
124         y_min = -1.5
125         y_max = +1.5
126
127         M = mandelbrot(x_min, x_max, nx, y_min, y_max, ny, ntasks)
128         tasks = M.get_tasks()
129
130         if size == 1:
131             for task in tasks:
132                 task.do_work()
133             TasksDoneByWorker = {0: len(tasks)}
134         else:
135             TasksDoneByWorker = manager(comm, tasks)
136
137         m = M.combine_tasks(tasks)
138         plt.imshow(m.T, cmap="gray", extent=[x_min, x_max, y_min, y_max])
139         plt.savefig("mandelbrot.pdf")
140
141         timespent += time.perf_counter()
142
143         print(f"Run took {timespent:f} seconds")
144         for i in range(size):
145             print(f"Process {i:5d} has done {TasksDoneByWorker.get(i, 0)
                    :10d} tasks")
146         print("Done.")
147
148     else:
149         worker(comm)
```

This listing implements a dynamic manager-worker algorithm. The process with rank zero is assigned the role of manager, while all other MPI processes are workers. The manager is responsible for creating the Mandelbrot tasks, distributing them to the workers, collecting the completed tasks, and finally assembling the global image. The workers do not communicate with each other; they only receive tasks from the manager and send completed results back to it.

The constants `TAG_TASK`, `TAG_TASK_DONE`, and `TAG_DONE` define the message types used in the program. Tags are important because the same pair of processes can exchange different kinds of messages. A message with tag `TAG_TASK` contains a new task to compute, a message with tag `TAG_TASK_DONE` contains a completed task returned by a worker, and a message with tag `TAG_DONE` tells a worker that no more tasks are available and that it can terminate.

Inside the `manager` function, the variable `next_task` stores the index of the next unassigned task. The variable `active_workers` counts how many workers are currently working on a task. At the beginning, the manager sends one task to each worker, as long as tasks are available. If there are more workers than tasks, the extra workers immediately receive a `TAG_DONE` message. This initialization phase fills the available workers with work and starts the computation.
After the initial distribution, the manager enters a loop that continues while at least one worker is active. The manager receives completed tasks using `comm.recv(source=MPI.ANY_SOURCE, tag=TAG_TASK_DONE, status=status)`. The use of `MPI.ANY_SOURCE` is essential for dynamic scheduling: the manager does not wait for a particular worker, but accepts the result from whichever worker finishes first. This allows faster workers or workers assigned easier Mandelbrot patches to immediately receive more work, improving load balance.
The received message contains both the `task_id` and the completed task object. The task identifier is needed because tasks may return in a different order from the one in which they were sent. The manager therefore uses `tasks[task_id] = completed_task` to put the result back into the correct position in the task list. This guarantees that the final call to `combine_tasks` can reconstruct the Mandelbrot image correctly.

Once a worker returns a result, the manager checks whether more tasks are available. If `next_task < len(tasks)`, the same worker immediately receives a new task. Otherwise, the worker receives a termination message with tag `TAG_DONE`, and the number of active workers is decreased. When all workers have received `TAG_DONE`, `active_workers` becomes zero and the manager loop terminates.

The `worker` function is a simple infinite loop. Each worker waits for a message from the manager using `comm.recv`. The tag of the received message is inspected through the `MPI.Status` object. If the tag is `TAG_DONE`, the worker exits the loop and terminates. Otherwise, the message contains a Mandelbrot task. The worker calls `task.do_work()`, which computes the Mandelbrot patch locally, and then sends the completed task back to the manager with tag `TAG_TASK_DONE`.

In the main part of the program, all MPI processes first obtain the communicator, the total number of processes, and their rank. The manager prints the number of MPI processes and creates the global Mandelbrot object. The call `M.get_tasks()` decomposes the global image into independent patches. If the program is run with only one MPI process, there are no workers. In this special case, the manager computes all tasks sequentially by directly calling `task.do_work()` for every task. This makes the code valid also for `size = 1` and provides a useful serial reference.
For `size > 1`, the manager calls the `manager` function to perform the dynamic parallel scheduling. After all tasks have been completed, `M.combine_tasks(tasks)` assembles the full Mandelbrot image from the computed patches.

Listing 16: Output of the manager-worker pattern for computing the Mandelbrot set in parallel

```
1 ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/03
     _ManagerWorker$ srun python3 manager_worker.py 4001 4001 100
2 MPI initialized with     4 processes
3 Run took 3.746471 seconds
4 Process     0 has done          0 tasks
5 Process     1 has done         33 tasks
6 Process     2 has done         34 tasks
7 Process     3 has done         33 tasks
8 Done.
```

The output shows a run with four MPI processes. Rank zero acts as the manager, while ranks one, two, and three are workers. The Mandelbrot domain has resolution `4001 x 4001` and is split into 100 tasks. Since the manager only schedules work and assembles the final result, process zero performs zero Mandelbrot tasks in this parallel run.
The tasks are distributed almost uniformly among the three workers: process 1 completes 33 tasks, process 2 completes 34 tasks, and process 3 completes 33 tasks. This indicates that the dynamic self-scheduling mechanism is working as intended. Workers request new work implicitly by returning completed tasks, and the manager assigns the next available task to whichever worker finishes first.

The near-equal number of completed tasks does not necessarily mean that all workers performed exactly the same amount of computational work, because different Mandelbrot patches may have different computational costs. Nevertheless, the dynamic scheduling strategy helps reduce idle time compared with a purely static

assignment. The total runtime printed by the manager includes the computation of the Mandelbrot patches, the communication of task objects between manager and workers, the collection of the completed patches, and the final assembly and output of the image.

2.3.2. Scaling study

We were asked to perform a scaling study using 2-32 workers for a 4001x4001 domain and split the workload into 50 and 100 tasks.
The scaling results for the manager-worker implementation of the Mandelbrot set are shown in figure 3.

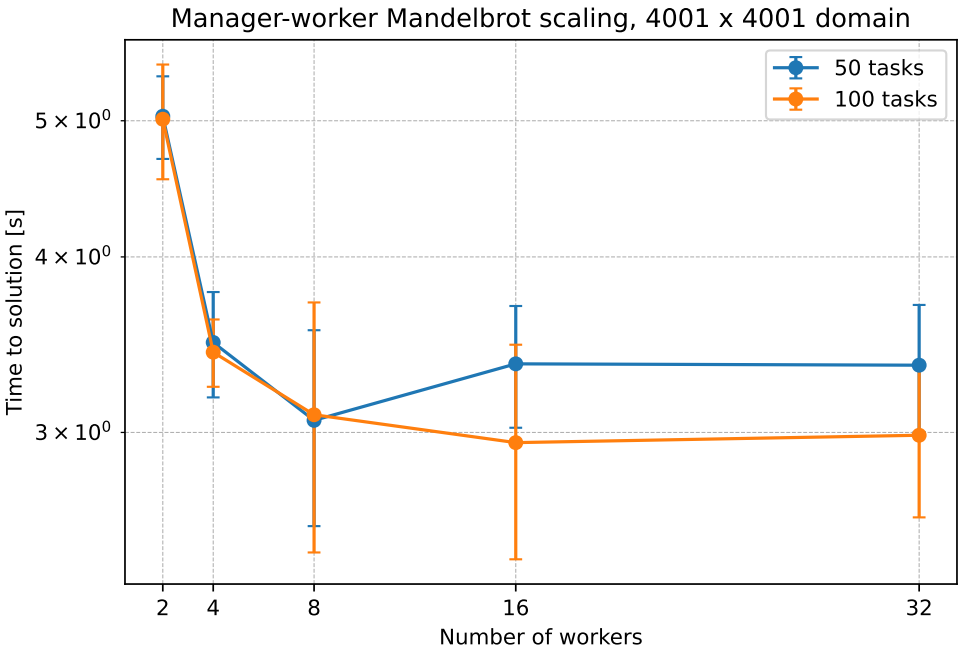


Figure 3: Scaling results for the manager-worker implementation of the Mandelbrot set.

Figure 3 shows the time to solution of the manager-worker Mandelbrot implementation for a fixed 4001×4001 domain, using 2, 4, 8, 16, and 32 workers. The workload is split into either 50 or 100 tasks.

The runtime decreases significantly when going from 2 to 4 workers. This shows that the manager-worker implementation is effective when the number of workers is still small: the Mandelbrot patches can be computed independently, and distributing them among more workers reduces the amount of work per worker. The improvement is also visible when going from 4 to 8 workers, although the gain is already smaller than in the previous step.

For larger worker counts, the scaling saturates. In particular, going from 8 to 16 and 32 workers does not lead to a clear additional reduction in runtime. For 50 tasks, the runtime even increases slightly after 8 workers. This indicates that the computation is no longer dominated only by the Mandelbrot calculation itself. The manager has to distribute tasks, receive completed patches, send termination messages, and finally assemble the global image. As the number of workers increases, the manager becomes a synchronization and communication bottleneck. Moreover, the number of tasks per worker becomes small, so the overhead of sending task objects and collecting results becomes comparable to the useful computation.

The comparison between 50 and 100 tasks shows the effect of task granularity. With 50 tasks, each task is larger, so the communication overhead per unit of computation is smaller. However, there are fewer tasks available for dynamic load balancing. This can limit scalability when many workers are used, because some workers may become idle while others are still working on expensive Mandelbrot patches. With 100 tasks, each task is smaller, so the manager has more flexibility to distribute work dynamically. This generally improves load balance, especially for higher worker counts, but it also increases the number of messages exchanged between the manager and the workers.

The results show that using 100 tasks gives slightly better performance at larger worker counts. In particular, for 16 and 32 workers the 100-task case is faster than the 50-task case. This suggests that, for this

problem size, the improved load balancing obtained with finer tasks compensates for the additional communication overhead. On the other hand, the difference between 50 and 100 tasks is not very large, which means that both decompositions are reasonably effective.

The error bars indicate the variability between repeated measurements. The variability is especially visible for larger worker counts. This is expected on a shared cluster system, where communication latency, node placement, and temporary load on the machine can affect the runtime. Since the measurements were performed with one process per node, the computation avoids intra-node resource contention, but it also makes the communication between manager and workers more expensive than a single-node run.

References

[1] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. 1st ed. CRC Press, 2010.